

Lexically Constrained and Interpretable Decoding for Neural Machine Translation

Alkım Gönenç Efe

Department of Computer Engineering

İzmir Katip Çelebi University

İzmir, Turkey

Abstract—Lexically constrained decoding allows users to inject domain-specific terminology into Neural Machine Translation (NMT) output at inference time, without retraining. While prior work has mostly followed structural approaches such as Grid Beam Search and Dynamic Beam Allocation, this paper takes a different route: adjusting raw output logits at each decoding step within a standard beam search pipeline. We implement and evaluate six strategies for English-to-Turkish (EN→TR) and Turkish-to-English (TR→EN) translation: hard exclusion, hard inclusion with dynamic anchor scheduling, soft penalty, curriculum-anchored soft reward with escalation, and two combined modes. This evaluation uses the Helsinki-NLP OPUS-MT models on a bilingual test set of 500 sentence pairs per direction. Hyperparameters are optimized separately per direction using Optuna. Soft reward with escalation achieves the best quality-satisfaction trade-off in both directions: 99.2%/98.8% satisfaction with Baseline BLEU of 72.32/81.65 and a near-natural length ratio of 1.062/1.058. Hard inclusion reaches 99.6%/99.2% satisfaction while running 2.1–3.3× faster than the HuggingFace Dynamic Beam Allocation (DBA) baseline. DBA’s structural beam allocator exhibits severe length inflation in EN→TR (length ratio 2.768), a direct consequence of agglutinative Turkish morphology forcing it to append words rather than place them inline. In TR→EN the same issue is far milder (1.289). We also report the *logit squeeze effect* in combined modes, the threshold suppression phenomenon in soft penalty, and the challenges of exact-match enforcement in agglutinative Turkish.

Index Terms—neural machine translation, lexical constraints, constrained decoding, logit manipulation, beam search, Turkish NLP, hyperparameter optimization

I. INTRODUCTION

Transformer-based neural machine translation (NMT) [1] produces fluent translations, but practical systems often require domain-specific terminology that the model would not otherwise generate. A medical system must use the approved drug name; a legal translation must match jurisdiction-specific terms. Retraining for each domain is expensive, and fine-tuning risks catastrophic forgetting.

Lexically constrained decoding addresses this by enforcing vocabulary constraints at inference time, without modifying model weights. Hokamp and Liu [2] introduced Grid Beam Search (GBS), which tracks constraint satisfaction across the beam. Post and Vilar [3] improved efficiency with Dynamic Beam Allocation (DBA), and Hu et al. [4] extended GBS to phrasal constraints. NeuroLogic Decoding [5] encodes constraints as logical predicates over the generation space.

Rather than altering the beam search structure, we explore *logit-level* enforcement: directly adjusting raw output logits

at each step within the standard HuggingFace Transformers pipeline [6]. The approach requires no changes to beam search internals and plugs into any model that exposes a LogitsProcessor interface.

We implement six decoding strategies (hard exclusion, hard inclusion with dynamic anchor scheduling, soft penalty, soft curriculum reward with escalation, and two combined modes) and evaluate them on a bilingual set of 500 EN↔TR sentence pairs per direction, split into HPO and held-out evaluation sets. Optuna [7] is used to optimize the logit pressure settings for each translation direction. We benchmark these methods against the HuggingFace DBA implementation under a morphology-aware configuration that provides it with full suffix and capitalization disjunctions. During evaluation we observed several practical issues, including the *logit squeeze* phenomenon in combined modes, threshold suppression in soft penalty, and exact-match failures in Turkish morphology.

II. RELATED WORK

A. Constrained Machine Translation

Existing constrained decoding methods fall into three categories: structural beam-search modifications, training-time terminology integration, and decoding-time control. Structural methods such as Grid Beam Search (GBS) [2] and Dynamic Beam Allocation (DBA) [3] modify the search algorithm to explicitly track constraint satisfaction states, guaranteeing terminology inclusion at the cost of execution complexity or beam allocation overhead. Phrasal variants [4] extend this coverage to multi-token spans. In contrast, terminology-aware training [8] bakes terminology tags directly into model parameters, teaching the network to align source terminology tags with target words. While training-time methods avoid decoding-time overhead, they cannot adapt dynamically to unseen terminology at inference time without retraining or fine-tuning.

B. Logit Manipulation and Decoding Control

Our approach operates entirely through logit manipulation during decoding. Modifying raw logits has been used to guide vocabulary selection in image captioning [9], steer search trajectories via gradient signals [10], or prevent generation degeneracy [11]. In free-text generation, methods like NeuroLogic decoding [5] resolve lexical constraints as logical predicates over candidates, and sample-based generators

[12] utilize Metropolis-Hastings steps. Unlike these methods, which require scoring completed sequences or altering standard beam states, our logit processors inject pressure directly at each decoding step. Furthermore, they address the subword segmentation and inflection boundaries typical of agglutinative translation.

C. Morphological Alignment and Tuning

Agglutinative target languages like Turkish present significant alignment challenges for subword tokenization schemes (e.g., BPE [13]) used in modern NMT models [14]. Suffix chains and phonetic variations mean a target lemma often mismatches the required exact-match surface constraints. While automated hyperparameter optimization is widely used to calibrate neural networks [7], we apply it here to balance the logit-level boundaries, escalation thresholds, and morphological penalties necessary to translate morphologically rich targets.

III. METHODOLOGY

A. System Architecture

We use the Helsinki-NLP OPUS-MT English–Turkish models [14], [15] implemented in Hugging Face Transformers [6]. Decoding uses 16-beam search [16] (maximum length 128, no-repeat 3-gram), with constraints applied through the LogitsProcessor interface before beam selection. The implementation is written in PyTorch [17].

An important preprocessing step is token expansion: each constraint word is tokenized into all plausible surface-form variants (capitalized, lowercase, with leading space, etc.). For Turkish inclusion constraints, we also enumerate common morphological suffixes to build a set of acceptable token sequences. Unlike English, where target words usually map one-to-one with surface forms, Turkish routinely concatenates grammatical roles directly onto the noun root. If the model is constrained to output the bare root but the context demands an accusative or dative suffix, exact-match enforcement will fail and force the word into unnatural syntactic positions. This expansion step partially addresses the agglutination mismatch discussed in Section II-C.

B. Hard Exclusion

Hard exclusion prevents forbidden tokens from ever appearing in the output. At each step, the processor sets the logits of all forbidden token IDs to $-\infty$, effectively removing them from the distribution:

$$l_t^{(i)} = -\infty \quad \forall i \in \mathcal{F}, \quad (1)$$

where $\mathcal{F}$ is the set of forbidden token IDs and $l_t^{(i)}$ is the logit for token i at step t . This is a simple, zero-overhead operation that guarantees 100% exclusion satisfaction without slowing down decoding.

C. Hard Inclusion with Dynamic Anchor Scheduling

Hard inclusion forces required words to appear in the output. The main challenge is that naively boosting a required token from step 1 disrupts sentence grammar: the model gets pushed toward a word before it has built up enough context for it to fit naturally.

Our processor addresses this with a *dynamic anchor schedule*. For each pending required word, the processor computes how much logit pressure to apply based on three signals: (1) whether the current step is within an “early token” grace period τ , (2) whether the required word’s natural rank in the beam is already within an acceptable threshold R_{sweet} , and (3) the current sentence completion ratio $\rho = t/(0.8 \cdot L_{\text{src}})$, where L_{src} is the source sentence length.

Formally, at step t , let r_t be the natural rank of the required token w in the current beam, and let $l_t^{(w)}$ be its logit. The applied boost δ_t is:

$$\delta_t = \begin{cases} 0 & \text{if } t \leq \tau \quad (\text{grace period}) \\ \beta_{\text{sweet}} & \text{if } r_t \leq R_{\text{sweet}} \quad (\text{natural fit}) \\ \max(0, l_t^* + A_{\text{start}} + A_{\text{range}} \cdot \rho - l_t^{(w)}) & \text{otherwise,} \end{cases} \quad (2)$$

where l_t^* is the maximum logit across all tokens, A_{start} and A_{range} are the anchor offset and range hyperparameters, and β_{sweet} is a small buffer applied when the word is already a high-probability candidate. The intuition is to delay aggressive intervention until the model has established enough syntactic context for the required word to fit naturally.

An EOS-blocking mechanism also prevents the model from ending the sentence while required words are still missing, unless the output has grown too long (beyond $1.5 \times L_{\text{src}}$ tokens), at which point a safety valve kicks in to stop runaway generation. Once a required multi-token sequence is placed, a morphological boundary penalty γ_{suffix} is applied to discourage the model from immediately suffixing the constraint word with agglutinative morphemes.

D. Soft Constraint Processor

The soft constraint processor adjusts logits continuously without making any hard guarantees. It has two components that run together.

Soft Penalty. For each penalty token $p \in \mathcal{P}$, the processor adds a fixed negative value $\lambda_{\text{pen}} < 0$:

$$l_t^{(p)} += \lambda_{\text{pen}} \quad \forall p \in \mathcal{P}. \quad (3)$$

This penalizes undesired words without strictly eliminating them.

Soft Curriculum Reward. For pending reward tokens, the processor implements curriculum anchoring [18]: the reward signal grows monotonically with the number of steps the reward has been active. Let η be the number of active reward steps and λ_{rew} be the base reward strength. The effective reward is:

$$\lambda_{\text{eff}} = \min(\lambda_{\text{max}}, \lambda_{\text{rew}} \cdot (1 + c \cdot \eta)), \quad (4)$$

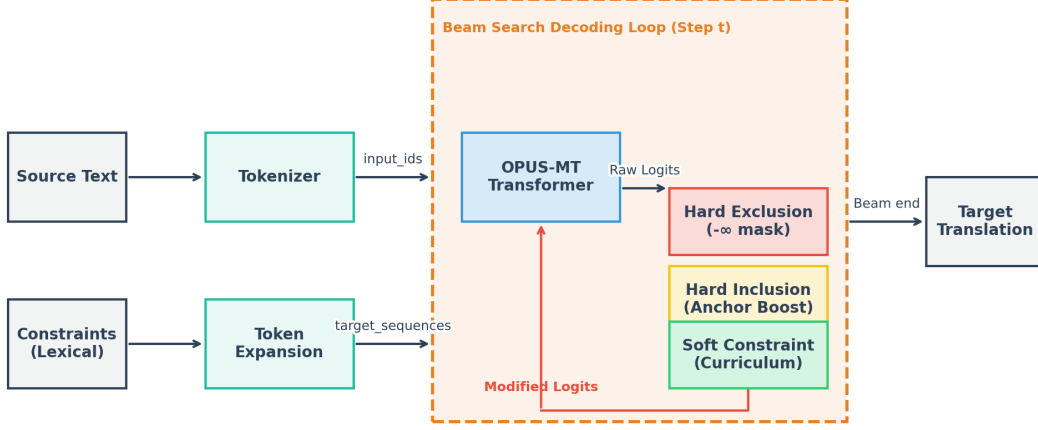


Fig. 1. System architecture. At each decoding step, active LogitsProcessor instances inspect the partially generated token sequence and modify the raw output logits before beam selection.

where c is the curriculum growth rate and $\lambda_{\max}$ caps the maximum pressure. To prevent early-token hallucination, reward application is suppressed for the first 3 decoding steps.

Rather than directly adding λ_{eff} as a delta, the processor uses a *target baseline* anchoring strategy: the reward token’s logit is pulled up to $l_t^* + \theta_{\text{offset}} + \lambda_{\text{eff}}$ if it is below this baseline, or receives a small contextual nudge ν if it is already above it. This ensures that the required token remains competitive with the beam’s top prediction without overwhelming the distribution.

E. Combined Strategies

Hard Combined. The hard combined mode runs both exclusion and inclusion processors at the same time. Exclusion runs before inclusion so that the anchor calculation ignores masked tokens. If the order were reversed, the anchor could be calibrated against a forbidden token’s high logit, over-boosting the required word.

Soft Combined. The soft combined mode runs both penalty and reward components of SoftConstraintProcessor in a single pass, followed by a soft-boost retry pass if reward words are still missing after the first generation.

F. Escalation Ladder (Soft Reward)

The `soft_reward_only` function uses a three-tier escalation ladder: (1) standard curriculum reward at the configured strength; (2) maximum-strength soft boost targeting only the words still missing; (3) hard inclusion as a last resort. The idea is to try the gentlest approach first and escalate only when needed. The downside is that difficult sentences may require up to three beam search passes.

G. Hyperparameter Optimization

We used Optuna [7] with the Tree-structured Parzen Estimator (TPE) sampler to find good parameter settings. Crucially, hyperparameters are optimized *separately* for each translation

direction on a dedicated HPO set, with the held-out evaluation set kept strictly separate.

For our proposed logit-level processors, we optimized a composite score that treats 1% of constraint satisfaction as equivalent to 2 BLEU points:

$$\text{score} = (S \times 200) + \overline{\text{BLEU}} - \mathcal{P}_{\text{len}}, \quad (5)$$

where $S \in [0, 1]$ is the constraint satisfaction rate, $\overline{\text{BLEU}}$ is the mean BLEU score relative to the unconstrained baseline, and $\mathcal{P}_{\text{len}}$ is a quadratic length penalty:

$$\mathcal{P}_{\text{len}} = \max(0, R_{\text{len}} - \theta)^2 \times 150,000, \quad (6)$$

with $\theta = 1.05$ for logit-level modes and $\theta = 1.03$ for DBA. R_{len} is the average output-to-unconstrained length ratio. The quadratic form aggressively penalizes even moderate length inflation: a ratio of 1.10 already contributes a penalty of 3,375, fully eliminating candidates that pad outputs to satisfy constraints rather than placing them inline.

Trial budgets reflect the complexity of each search space: 50 trials for hard inclusion, hard combined, and soft combined (7 and 5 parameters with complex interactions); 30 trials for DBA (3 parameters); 10 trials for soft penalty (1 parameter with a flat landscape). The optimized hyperparameters for both directions are shown in Table I.

IV. EXPERIMENTAL SETUP

A. Models and Hardware

All experiments use the Helsinki-NLP OPUS-MT “big” transformer models: `opus-mt-tc-big-en-tr` for English-to-Turkish and `opus-mt-tc-big-tr-en` for Turkish-to-English [14], [15]. These are 6-layer encoder-decoder transformers with 8 attention heads and a model dimension of 512. Models are downloaded and cached locally. We ran everything on a single NVIDIA GPU with CUDA support, using PyTorch [17] in no-gradient inference mode.

TABLE I
OPTIMIZED HYPERPARAMETERS FOUND BY OPTUNA TPE (50/10/30 TRIALS PER MODE). ALL VALUES ARE PER-DIRECTION; EN→TR TARGETS TURKISH, TR→EN TARGETS ENGLISH. PARAMETERS MARKED WITH † SHOW THE LARGEST DIRECTION ASYMMETRY.

Group	Parameter	EN→TR	TR→EN	Role / Interpretation
Hard Inclusion	Early-token grace $\tau^\dagger$	0	5	Steps before any boost is applied. Turkish targets benefit from immediate pressure ($\tau=0$); English targets improve with a short passive window.
	Sweet-rank threshold $R_{\text{sweet}}^\dagger$	557	293	If the required word ranks below this in the beam, only a light buffer is applied instead of the full anchor boost.
	Sweet-rank buffer $\beta_{\text{sweet}}^\dagger$	12.27	5.86	Logit boost applied when the word is already within the sweet-rank window (natural fit).
	Anchor start A_{start}	-6.55	-7.50	Anchor target at 0% sentence completion, expressed as offset below the current max logit.
	Anchor range A_{range}	10.31	9.83	Total logit climb of the anchor target from 0% to 100% completion.
	Inclusion boost	5.74	7.18	Fixed logit boost applied when the required token is placed by the hard processor.
Soft Reward	Suffix penalty $\gamma_{\text{suffix}}^\dagger$	-21.92	-9.34	Penalty applied immediately after a constraint word is placed to discourage agglutinative suffix attachment (stronger in Turkish targets).
	Reward strength λ_{rew}	2.03	2.01	Base curriculum reward added to the required token's logit at each step.
	Reward cap λ_{max}	15.37	19.04	Maximum reward that the curriculum schedule can reach.
	Curriculum rate c	2.96	0.17	Controls how quickly the reward grows with active steps.
	Anchor offset $\theta_{\text{offset}}^\dagger$	-23.89	-28.74	Reward baseline offset below the current max logit. Values $\lesssim -15$ create a passive safety net; values near 0 force early injection and hurt BLEU.
Soft Penalty	Penalty λ_{pen}	-98.37	-118.52	Any value $\lesssim -30$ produces identical results (threshold suppression); exact magnitude is not critical.
DBA Baseline	Beam size †	7	5	More beams needed in EN→TR for the allocator to encounter Turkish surface forms.
	Length penalty †	+0.01	-4.61	Near-zero in EN→TR (length cannot be controlled); strongly negative in TR→EN to curb residual appending.
	Repetition penalty	1.03	1.20	Discourages DBA's tendency to repeat tokens or punctuation in appended suffixes.

B. Test Set

We built a bilingual evaluation set of 500 sentence pairs (250 EN→TR, 250 TR→EN) derived from the Tatoeba English–Turkish test split [19]. Constraints are generated automatically by comparing each human reference against the unconstrained OPUS-MT baseline: words present in the reference but missing from the baseline become *required* constraints, and vice versa for *forbidden* constraints. We check morphological overlap to avoid penalizing relatives of required words. Cases are split evenly between *easy* (baseline nearly produces the word) and *hard* (baseline omits it entirely) instances, then divided into dedicated HPO and held-out evaluation sets.

Each case specifies forbidden/required words for hard modes, and corresponding penalty/reward words for soft modes. The baseline translation is computed once per sentence. Exclusion-only modes are evaluated on the 163 EN→TR and 142 TR→EN cases with exclusion annotations; inclusion-bearing and combined modes use the 250 EN→TR and 247 TR→EN cases with required-word annotations.

C. Evaluation Metrics

We measure each mode on four fronts. *Constraint Satisfaction Rate (S)*: the fraction of test cases where all required words appear in the output (inclusion) and no forbidden words appear (exclusion), checked with a morphology-aware matcher that accepts valid Turkish inflectional suffixes. *Baseline BLEU*: computed against the unconstrained baseline output, so it captures how much the constraint changes the output rather than absolute quality. For instance, a Baseline BLEU of 100 means the constraint had no effect. *Reference BLEU* [20] and *ChrF* [21]: both computed against human ground-truth reference translations, measuring absolute translation quality. *Length Ratio*: average constrained-to-unconstrained output length, which serves as a quick diagnostic for word-appending artifacts.

D. Evaluation Protocol

All reported results are averaged over three random seeds (42, 123, 7) to reduce stochastic noise in the pipeline. For the DBA baseline, we use HuggingFace's

`DisjunctiveConstraint` to encompass all suffix and capitalization variants for required words, and provide a complete `flat_token_ids` map for `bad_words_ids`, representing a maximally generous configuration.

V. RESULTS AND DISCUSSION

A. Aggregate Results

Tables II and III summarize the aggregate metrics for EN→TR and TR→EN respectively, averaged over three seeds. We report Baseline BLEU, Reference BLEU, ChrF, decoding latency, and the average number of generation passes. The `hard_inclusion_ablation` mode (identical code path to `hard_inclusion`, included as a pipeline sanity check) confirmed matching results and is omitted here for brevity.

B. Hard Constraint and DBA Baseline Analysis

Hard exclusion achieves 98.2% satisfaction in EN→TR and 99.3% in TR→EN, with very low length inflation in both directions (ratios of 1.009 and 1.016). The model rephrases to avoid forbidden terms without significant elongation.

Hard inclusion achieves 99.6%/99.2% satisfaction in EN→TR/TR→EN, representing improvements over the previous single-direction figure of 97.6%. Baseline BLEU scores show direction asymmetry: 50.49 in EN→TR versus 73.93 in TR→EN. This gap reflects the fundamentally harder constraint problem in agglutinative Turkish targets, where the model must place a required word while also managing suffix attachment and morphophonological agreement, leaving less room for natural phrasing.

The HuggingFace DBA baseline reveals a critical direction-dependent failure. In EN→TR, DBA reaches only a 2.768 average length ratio, meaning outputs are nearly *three times* the expected length. The structural beam allocator, unable to find valid Turkish surface forms among its beam hypotheses, resorts to appending constraint words at the end of otherwise complete translations, often entering repetitive punctuation loops. Qualitative inspection of EN→TR DBA outputs confirms strings such as “... değil mi?... b... ???!?? ? ???... ??” appended after legitimate translations. The Baseline BLEU of 31.17 for EN→TR DBA is therefore a consequence of this length explosion rather than a direct quality failure. The underlying translation up to the appended portion is often reasonable.

In TR→EN, the picture is considerably better: DBA achieves a 1.289 length ratio and 97.2% satisfaction. English targets, being morphologically simple, allow DBA to find constraint words within the natural beam hypotheses without resorting to appending. This direction-asymmetric behavior limits DBA’s practical applicability for EN→TR in our setting.

In contrast, our hard inclusion maintains a 1.063/1.062 length ratio in both directions and runs $2.1\times$ faster than DBA in EN→TR (943.3 vs. 2003.3 ms) and $3.3\times$ faster in TR→EN (239.9 vs. 784.0 ms). Fig. 4 and Fig. 5 make these comparisons explicit.

C. Combined and Soft Constraint Analysis

Hard combined enforces exclusion and inclusion simultaneously, but shows non-linear degradation (the *logit squeeze effect*) below what either isolated mode achieves: EN→TR reaches 91.2% satisfaction with 32.53 Baseline BLEU, while TR→EN reaches 96.0% satisfaction with 56.55 Baseline BLEU. This asymmetry appears to stem from the fact that Turkish exclusion masks more token IDs per forbidden word (since agglutinative morphology generates more surface variants), narrowing the beam distribution aggressively and leaving less room for the inclusion anchor to operate.

For soft constraints, soft penalty produces results that are numerically identical to hard exclusion: the same satisfaction rates, BLEU scores, and length ratios. This stems from *threshold suppression* in beam search, where penalties sufficiently negative to push a forbidden token out of the top- K beam effectively mimic a hard $-\infty$ mask. Under our configuration, soft penalty offered no measurable advantage over hard exclusion. Soft reward with escalation achieves the best overall trade-off: 99.2%/98.8% satisfaction and 72.32/81.65 Baseline BLEU, with the escalation ladder activating on roughly 40% of sentences. Soft combined drops satisfaction to 62.0%/71.3% due to competing penalty and reward forces.

Fig. 2 shows the full quality breakdown across all modes for both directions.

D. Dynamic Scheduling and Hyperparameter Sensitivity

Fig. 3 shows the dynamic anchoring schedule simulated using the optimized parameters. The EN→TR schedule applies immediate pressure ($\tau = 0$) starting at $A_{\text{start}} = -6.55$ and climbing by $A_{\text{range}} = 10.31$. Conversely, TR→EN uses $\tau = 5$ grace steps, $A_{\text{start}} = -7.50$, and $A_{\text{range}} = 9.83$. This difference suggests that English targets benefit from immediate constraint pressure, whereas Turkish targets require a short passive window to establish syntactic structure.

Hyperparameter optimization (HPO) studies reveal distinct structural sensitivities. Soft penalty is one-shot: any value more negative than roughly -30 pushes tokens outside the top- K beam, making search optimization unnecessary. Soft reward, conversely, exhibits a large flat optimum. Because the reward baseline only intervenes when the required word’s probability drops below a safe threshold, offsets $\lesssim -15$ act as a passive safety net. Once this anchor becomes sufficiently passive, exact parameter values matter very little; the model maintains its natural fluency and only receives a targeted boost when it genuinely threatens to omit the constraint word. Hard inclusion displays a much more highly sensitive landscape across its seven parameters. Because it must forcefully inject tokens while simultaneously calculating anchor climbs and suffix penalties, it requires the full 50-trial budget to carefully calibrate these cooperating settings. DBA tuning is similarly asymmetric, converging to 7 beams with near-zero length penalty in EN→TR to force Turkish surface forms into the beam, and 5 beams with negative penalty in TR→EN to check length inflation.

TABLE II
AGGREGATE RESULTS — ENGLISH TO TURKISH (EN→TR, $N_{\text{EXCL}} = 163$, $N_{\text{INCL}} = 250$)

Mode	N	Sat. %	BLEU (Base)	BLEU (Ref)	ChrF (Ref)	Len. Ratio	Latency (ms)	Passes
Unconstrained	250	—	100.00	40.97	67.04	1.000	96.9	1.00
Hard Exclusion	163	98.2%	35.20	35.59	58.84	1.009	165.2	1.00
Hard Inclusion	250	99.6%	50.49	31.11	65.88	1.063	943.3	1.00
Hard Combined	250	91.2%	32.53	34.98	65.04	1.114	1082.6	1.00
Soft Penalty	163	98.2%	35.20	35.59	58.84	1.009	167.9	1.00
Soft Reward (Escal.)	250	99.2%	72.32	40.98	71.31	1.062	838.0	1.79
Soft Combined	250	62.0%	57.65	47.62	68.53	1.007	386.6	1.37
HuggingFace DBA	250	97.6%	31.17	34.41	62.39	2.768	2003.3	1.00

TABLE III
AGGREGATE RESULTS — TURKISH TO ENGLISH (TR→EN, $N_{\text{EXCL}} = 142$, $N_{\text{INCL}} = 247$)

Mode	N	Sat. %	BLEU (Base)	BLEU (Ref)	ChrF (Ref)	Len. Ratio	Latency (ms)	Passes
Unconstrained	250	—	100.00	49.55	66.99	1.000	101.2	1.00
Hard Exclusion	142	99.3%	44.74	40.58	61.52	1.016	188.4	1.00
Hard Inclusion	247	99.2%	73.93	44.86	69.57	1.062	239.9	1.00
Hard Combined	247	96.0%	56.55	51.07	72.66	1.050	399.4	1.00
Soft Penalty	142	99.3%	44.74	40.58	61.52	1.016	184.6	1.00
Soft Reward (Escal.)	247	98.8%	81.65	49.07	71.65	1.058	314.5	1.71
Soft Combined	247	71.3%	66.52	54.75	71.62	1.013	236.8	1.32
HuggingFace DBA	250	97.2%	49.08	50.38	70.36	1.289	784.0	1.00

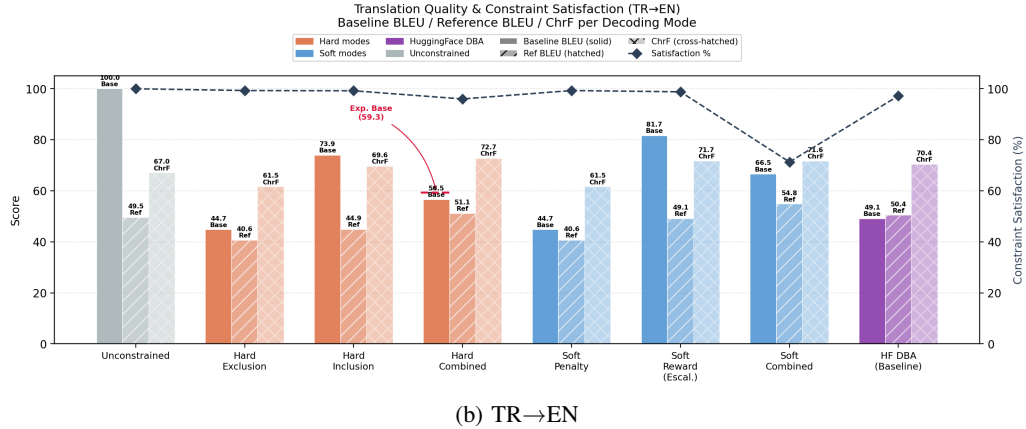
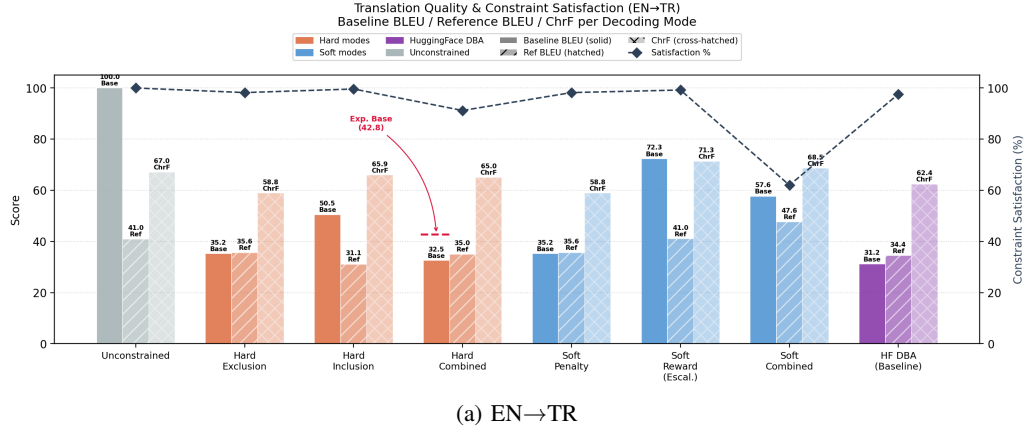


Fig. 2. Translation quality (Baseline BLEU / Reference BLEU / ChrF) and constraint satisfaction per decoding mode, for both translation directions.

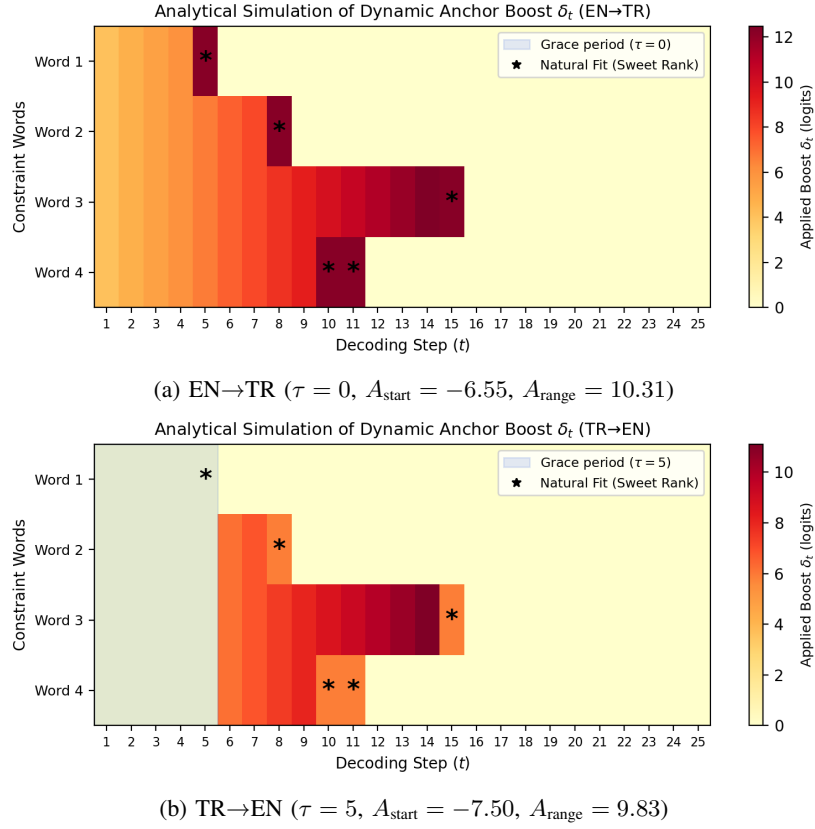


Fig. 3. Dynamic anchor schedule for each direction. Color intensity indicates applied logit boost δ_t . The grace period (zero boost), sweet-rank windows (light buffer), and full anchor pressure are distinguishable. The EN→TR direction applies pressure from step 1; TR→EN waits five steps.

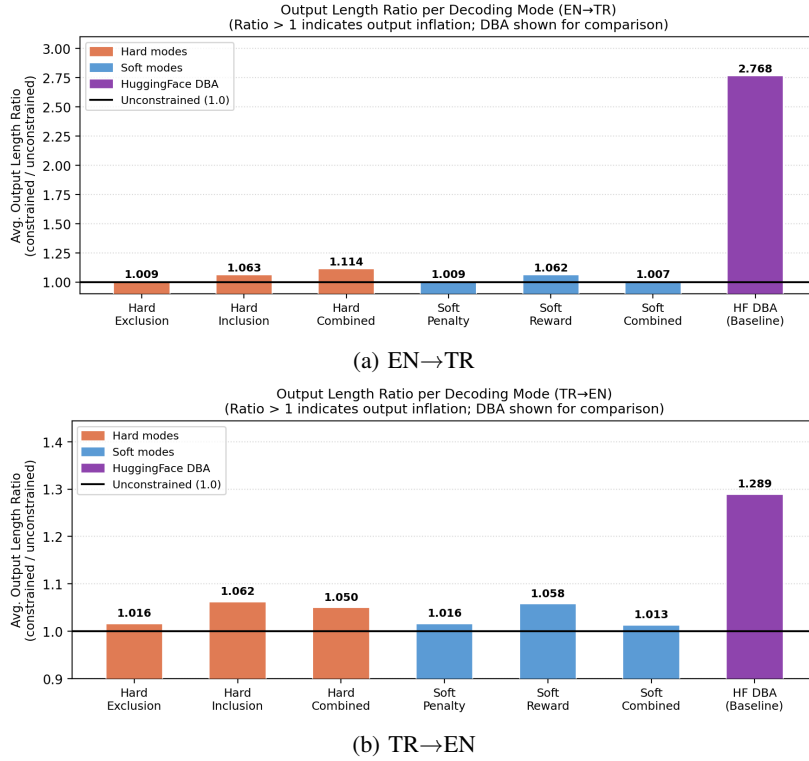
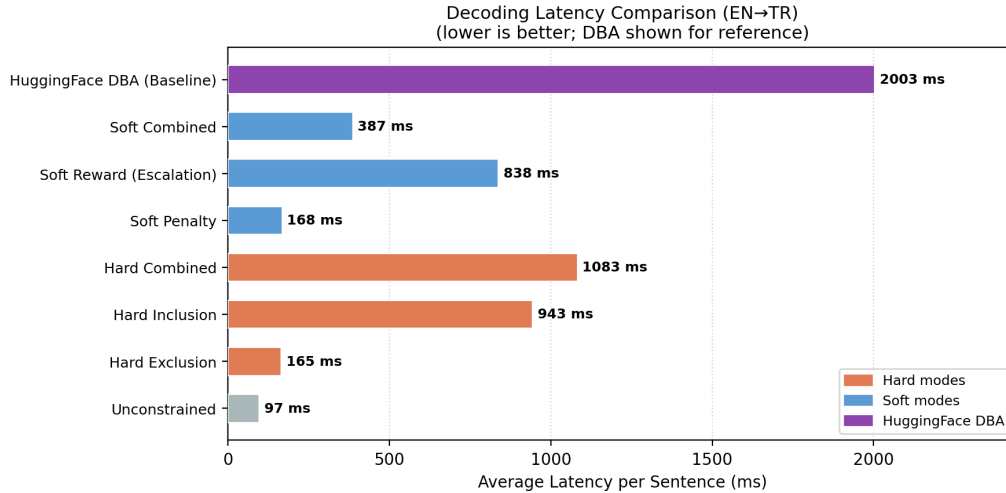
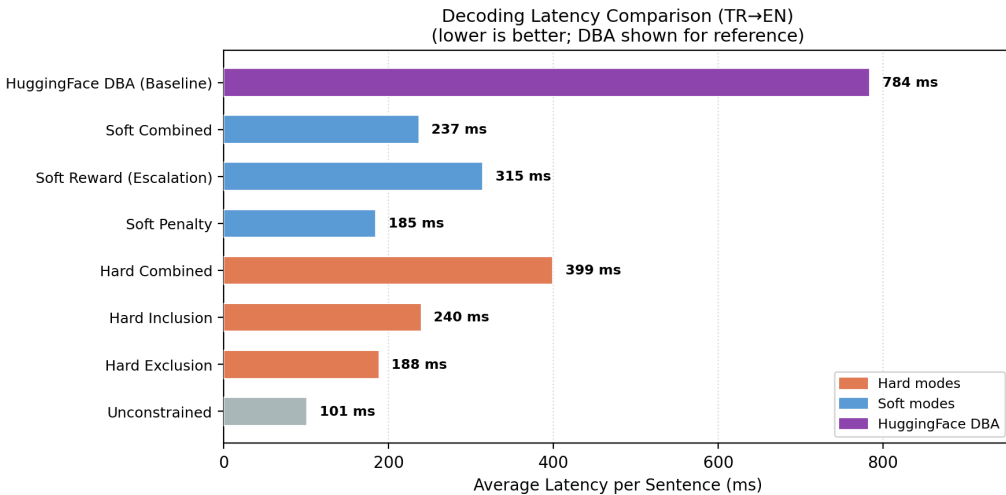


Fig. 4. Output length ratio per decoding mode. DBA's length inflation is severe in EN→TR (2.768) due to agglutinative Turkish morphology, and moderate in TR→EN (1.289). All logit-level modes remain below 1.12 in both directions.



(a) EN→TR



(b) TR→EN

Fig. 5. Average decoding latency per sentence for all modes and the DBA baseline. Logit-level methods run well under 1000 ms; DBA requires over 2000 ms in EN→TR and 784 ms in TR→EN.

E. Qualitative and Morphological Analysis

Qualitative inspection of hard inclusion failures highlights three limitations: (1) competing schedules for multiple constraints resulting in repetition (e.g., repeating *uzay istasyonu* instead of producing both *uzay istasyonu* and *yörünge*); (2) semantic mismatch where syntax templates lock in baseline synonyms (e.g., *movies* instead of *cinema*); and (3) ungrammatical insertion from simultaneous logit pressure (e.g., *The little dog tiny barked*).

Furthermore, word-appending artifacts in Turkish stem from the agglutination mismatch. When the constraint is a bare root (e.g., *hekim*) but the model prefers an inflected form (e.g., *hekimin*), exact-match checks fail, and EOS-blocking appends the bare root at the end. Suffix expansion and suffix penalties address this, yielding a 99.6% EN→TR satisfaction rate, but complex suffix chains remain difficult to cover fully.

F. Methodological Transparency

We note three main limitations. First, while the corpus is split into HPO and evaluation sets, both are drawn from the same curated bilingual source, so some distributional overlap remains. Second, the soft reward mode’s strong results require up to three beam search passes per sentence, which may not be practical for latency-sensitive applications. Third, Reference BLEU and ChrF are computed against a single human reference per sentence and may not fully reflect output quality for sentences with many equally valid translations.

G. Conclusion

Across both translation directions, curriculum-based soft reward provides the best balance between lexical satisfaction and translation quality, while hard inclusion offers a faster alternative to DBA without its severe length inflation in

English→Turkish. These results suggest that logit-level control can compete with structural constrained decoding while requiring far fewer modifications to existing decoding pipelines. Future research will explore joint training of constraint processors and morphology-aware constraint metrics specifically tailored for agglutinative target languages.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [2] C. Hokamp and Q. Liu, “Lexically constrained decoding for sequence generation using grid beam search,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2017, pp. 1535–1546.
- [3] M. Post and D. Vilar, “Fast lexically constrained decoding with dynamic beam allocation for neural machine translation,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018, pp. 1314–1324.
- [4] J. E. Hu, H. Khayrallah, R. Murray, P. McNamee, M. Post, and M. Dreyer, “Improved lexically constrained decoding for translation and monolingual rewriting,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2019, pp. 839–850.
- [5] X. Lu, P. West, R. Zellers, R. Le Bras, C. Bhagavatula, and Y. Choi, “NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 4288–4299.
- [6] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45.
- [7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631.
- [8] G. Dinu, P. Mathur, M. Federico, and Y. Al-Onaizan, “Training neural machine translation to apply terminology constraints,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 3063–3068.
- [9] P. Anderson, B. Fernando, M. Johnson, and S. Gould, “Guided open vocabulary image captioning with constrained beam search,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2017, pp. 936–945.
- [10] L. Sha, “Gradient-based decoding for lexical constraints: Ensuring word presence with flexible integration,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 3974–3983.
- [11] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [12] N. Miao, H. Zhou, L. Mou, R. Yan, and L. Li, “CGMH: Constrained sentence generation by metropolis-hastings sampling,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 6834–6842.
- [13] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2016, pp. 1715–1725.
- [14] J. Tiedemann and S. Thottingal, “OPUS-MT – building open translation services for the world,” in *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*. European Association for Machine Translation, 2020, pp. 479–480.
- [15] J. Tiedemann, M. Aulamo, B. Bam, S. Doddapaneni, S.-A. Grönroos, T. Nieminen, A. Raganato, U. Remes, Y. Scherrer, and S. Virpioja, “Democratizing machine translation with OPUS-MT,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, 2023, pp. 517–525.
- [16] M. Freitag and Y. Al-Onaizan, “Beam search strategies for neural machine translation,” in *Proceedings of the First Workshop on Neural Machine Translation*. Association for Computational Linguistics, 2017, pp. 56–60.
- [17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.
- [18] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning*. ACM, 2009, pp. 41–48.
- [19] J. Tiedemann, “The Tatoeba translation challenge – realistic data sets for low resource and multilingual MT,” in *Proceedings of the Fifth Conference on Machine Translation*. Association for Computational Linguistics, 2020, pp. 1174–1182.
- [20] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.
- [21] M. Popović, “chrF: character *n*-gram F-score for automatic MT evaluation,” in *Proceedings of the Tenth Workshop on Statistical Machine Translation*. Association for Computational Linguistics, 2015, pp. 392–395.